

4.1 Introduction to functions

A Function is a self-contained block of statement that performs a specific task when called. A function is also called as sub program or procedure or subroutine.

Every C Program can be thought of as a collection of these functions. Function is a set of instructions to carry out a particular task. Function after its execution returns a single value. Generally, the functions are classified into two types:

1. Standard functions
 2. User-defined functions.
- ✓ The Standard functions are also called *library functions* or *built-in functions*.
 - ✓ All standard functions, such as *printf()*, *scanf()*, *sqrt()*, *abs()*, *log()*, *sin()* etc. are provided in the library of function.
 - ✓ Most of the applications need other functions than those are not available in the software, those are known as user-defined functions.

Advantages of functions

Several advantages of dividing the program into functions include:

1. Reduce the complexity of the program.
2. Modularity
3. Reduction in code redundancy
4. Enabling code reuse
5. Better readability.

Parts of a function

A function has the following parts:

1. Function prototype declaration.
2. Function definition.
3. Function call.
4. Actual arguments and Formal arguments.
5. The *return* statement.

Function Declaration

Function prototype declaration consist function return type, function name, and argument list. A function must be declared before it is used

Syntax

```
return_type function_name(parameter_list);
```

parameter list: type param1, type param2, type param3, etc

Examples

```
double sqrt(double);  
int func1();  
void func2(char c);
```

Function Definition also known as function implementation, means composing a function.

Every Function definition consists of two Parts:

1. Header of the function,.
2. Body of the function.

Syntax

```
return_type function_name(parameter_list)  
{  
    // Function body  
}
```

The body of a function consists of a set of statements enclosed within braces. The *return* statement is used to return the result of the computations done in the called function and/or to return the program control back to the calling function. A function can be defined in any part of the program text or within a library

Example:

```
void welcome( )  —————> function Header
{
    printf("welcome to CSE \n");
}                } function body
```

Function Call

A function call has the following syntax:

<function name>(<argument list>);

Example:

sum(a,b);

4.2 Types of functions

Functions can be divided into 4 categories:

1. A function with no arguments and no return value
2. A function with no arguments and a return value
3. A function with an argument or arguments and returning no value
4. A function with arguments and returning a values

1. A function with no arguments and no return value

- ✓ Called function does not have any arguments
- ✓ Not able to get any value from the calling function
- ✓ Not returning any value
- ✓ There is no data transfer between the calling function and called function.

Example

```
#include<stdio.h>
void welcome();           //function prototype declaration
int main()                // calling function
{
    welcome(); //function call
    return 0;
}
void welcome()             //called function
{
    printf("Hi students..!");
}                          } function body
```

OUTPUT
Hi students..!

2. A function with no arguments and a return value

- ✓ Does not get any value from the calling function
- ✓ Can give a return value to calling program

Example

```
#include <stdio.h>
int send();
int main()
```

```
{
    int x;
    x=send()
    printf("\nYou entered : %d",x);
    return 0;
}
int send()
{
    int n;
    printf("\nEnter a number: ");
    scanf("%d",&n);
    return n;
}
```

OUTPUT

Enter a number: 24
You entered: 24

3. A function with an argument(s) and returning no value

- ✓ A function has argument(s).
- ✓ A calling function can pass values to function called , but calling function not receive any value.
- ✓ Data is transferred from calling function to the called function but no data is transferred from the called function to the calling function.
- ✓ Generally Output is printed in the Called function.

Example

```
#include<stdio.h>
void add(int x, int y);
int main()
{
    add(24,18);
    add(124,100);
    add(1224,77);
    return 0;;
}
void add(int a, int b)
{
    int result;
    result = a+b;
    printf("Sum of %d and %d is %d\n",a,b,result);
}
```

OUTPUT

Sum of 24 and 18 is 42
Sum of 124 and 100 is 224
Sum of 1224 and 77 is 1301.

4. A function with arguments and returning a values

- ✓ Argument are passed by calling function to the called function
- ✓ Called function return value to the calling function
- ✓ Data returned by the function can be used later in our program for further calculation
- ✓ Mostly used in programming because it can two way communication

Example

```
#include <stdio.h>
int add(int x, int y);
int main()
{
    int z;
```

```
        z=add(24,18);
        printf("Result %d\n", add(124,76));
        printf("Result %d\n", z);
        return 0;
    }
    int add(int a, int b)
    {
        int result;
        result = a + b;
        return (result);
    }
```

OUTPUT

Result = 300
Result = 42

Note:

- ✓ If the function declaration doesn't specify return type, then the default return type is integer value.

Example:

```
#include<stdio.h>
add(int a); // in function declaration we are not specifying the return type
int main()
{
    printf("the value of x is:%d",add(24));
    return 0;
}
int add(int x) // in definition we place the int return type because int is default.
{
    return x;
}
```

- ✓ The parameter list in the function definition and function declaration must match.
- ✓ A logical error will be generated if the arguments in the function call are placed in the wrong order.

```
#include<stdio.h>
void student(int sid, char sname[10],float smarks,char grade);
int main()
{
    // student(1224,8.8,"Ram",,'A'); - → placed in wrong order.
    student(1224,"Ram",8.8,'A');
    return 0;
}
void student(int sid, char sname[10],float smarks,char grade)
{
    printf("Name:%s\t Idno:%d\tMarks:%f\tGrade:%c",sname,sid,smarks,grade);
}
```

- ✓ The programmer may or may not place the expression in a return statement within parenthesis.

return a+b; **or** return (a+b);

4.3 Storage classes

The storage classes based on four parameters they are

1. Storage
2. Initial value
3. Scope (Scope of variable)
4. Life time of variable

1. Storage : (Where the variable would be stored)

From C compiler's point of view, a variable name identifies some physical location within the computer where the value of the variable is stored. There are basically two kinds of locations in a computer where such a value may be kept—

1. Memory (**RAM**)
2. CPU registers.

2. Initial value :

It specifies whether the variable will be automatically initialized to zero or unpredictable value (Garbage value), if initial value is not specifically assigned (i.e the default value).

3. Scope of the Variables :

- ✓ What is the scope of the variable; i.e. in which functions the value of the variable would be available.
- ✓ All variables have a defined scope, by scope we mean the **accessibility** and **visibility** of variables at different point in the program.
- ✓ There are four types of scope:
 - a) Block Scope.
 - b) Function / Method Scope.
 - c) Program Scope.
 - d) File Scope.

a) Block Scope:

- ✓ A block is a group of statements enclosed with opening and closing curly brackets {}.
- ✓ If a variable declared within the block then we access that variable within that particular block only we can't access that variable outside the block, such variables are called block scope variables. It is also known as Local variable i.e. declare a variable inside the function is called local variable.
- ✓ The scope the variable access particular block where the variable declared.

Example :

```
#include<stdio.h>
void main()
{
    {
        int a=10;
        printf("the value of A is :%d",a); → here prints a value is 10;
    }
    printf("the value of A is :%d",a); → here prints an error that is a is not declared in this function
}                                     because a is defined inside of another block, we can't
                                     Access that variable outside the block.
```

Example 2:

```
#include<stdio.h>
void main()
{
```

```

int a=24;
int i=1;
printf("the value of outside the block is %d",a); → prints 24
while(i<4)
{
    int a=i;
    printf("the value of inside the block is %d",a); → prints 1 2 3
    i++;
}
printf("the value of outside the block is %d",a); → prints 24
}

```

Note: In the above program we declared and initialized an integer variable 'a' in main() function and then re-declared and re initialized in a while loop, then the variable 'a' will be considered as two different variables and occupy different memory slots.

b) Function / Method Scope:

- ✓ A variable declare inside the function, We can access a variable starting to the end of a function is known as function scope.
- ✓ We can also access a variable inside blocks or loops but you should not declare a variable with same name then only we can access.
- ✓ This variable also known as a local variable.

Example 1: with function scope and block scope variables.

```

#include<stdio.h>
void main()
{
    int a; //function scope
    {
        int a=10; // block scope
        printf("%d",a); → prints 10
    }
    printf("%d",a); → prints garbage value
    test( );
}

void test( )
{
    printf("%d",a); → Error the function
                        scope variable we can't
                        access outside the
                        function, here the error
                        a is not declared.
}

```

Note : In this program, function scope and block scope variable both are different variable with same name.

Example 2: function scope without block scope variables.

```

#include<stdio.h>
void main()
{
    int a; //function scope
    {
        a=10;
        printf("%d",a); → prints 10
    }
    printf("%d",a); → prints 10
}

```

OUTPUT: 10 10

Note: In this program we are not declared block scope variable then we access a function scope variable inside the block because the block is declared inside the function. If we declared a variable with same name of function scope variable inside the block then we can't access the function scope variable. Block scope variable is overrides the function scope variable.

c) Program Scope:

- ✓ A variable declare outside the function ,We can access a variable in entire program is known as program scope.
- ✓ It is also known as global variable. If we declare a variable outside the function is known as global variable.
- ✓ Global variable are not limited to a particular function so they exists even when a function calls another functions. These variables can be used from every function in the program.
- ✓ The global variables are declared outside the main() function.
- ✓ If we have a variable declared in a function that as same name as global variable, then the function will use the local variable declared within it and ignore the global variable.

Example 1:**Program scope with function and block scope variables**

```
#include<stdio.h>
int a=10; // program scope
void test( );
void main( )
{
    int a=20; //function scope
    {
        int a=30; // Block scope
        printf("\n value of A inside the block
                scope :%d",a);
    }
    printf("\nvalue of A inside the function
            scope :%d",a);
test( );
}

void test( )
{
    printf("\nvalue of A in other functions:%d",
            a);
}
```

OUTPUT:

```
value of A inside the block scope:    30
value of A inside the function scope: 20
value of A inside in other functions: 10
```

Note:

- ✓ If we not initialized global variable values then compiler places the default integer value '0'.
- ✓ In above program the function and block scope 'a' variables overrides global variable 'a'.

Example 2:**Program scope without function and block scope variables.**

```
#include<stdio.h>
int a=10; // program scope
void test( );
void main( )
{
    a=20;
    {
        a=30;
        printf("\nvalue of A inside the block
                scope :%d",a);
    }
    printf("\nvalue of A inside the function
            scope :%d",a);
test( );
}

void test( )
{
    printf("\nvalue of A in other functions:%d",
            a);
}
```

OUTPUT:

```
value of A inside the block scope:    30
value of A inside the function scope: 30
value of A inside in other functions: 30
```

Note:

- ✓ In the above program we declare only global variable, but initialized value to global variable in function and block scopes.

d) File Scope :

When a global variable is accessible until the end of the file, the variable said to have file scope. Declare variable with the static keyword.

Ex : static int a=10;

4. Life time of variable: (how long would the variable exist)

- ✓ The life time of any variable is the time for which the particular variable outlives in memory during the running of the program.

4.3.1 Types of Storage Classes

There are four storage classes in C:

1. auto
2. register
3. static
4. external

Automatic Variables

These are declared inside a function in which they are to be utilized. These are declared using a keyword *auto*.

Example:

auto int number;

- ✓ These are created when the function is called and destroyed automatically when the function is exited.
- ✓ These variables are private (local) to the function in which they are declared.
- ✓ Variables declared inside a function without storage class specification is, by default, an automatic variable. i.e int a =10; and auto int a=10; both are same.

Properties of automatic variable are as under:

Storage	Memory.
Initial value	An unpredictable value, which is often called a <i>garbage value</i> .
Scope	Local to the block in which the variable is defined
Life	Till the control remains within the block in which the variable is defined.

Example 1:

```
#include<stdio.h>
void main( )
{
    auto int a=10;
    {
        auto int a;
        printf("the value inside block:%d\n",a);
    }
    printf("the value outside block:%d\n",a);
}
```

OUTPUT:

the value inside block: **Garbage value**
the value outside block: **10**

Example 2:

```
#include<stdio.h>
int main( )
{
    auto int i=1;
    {
        auto int i=2;
        {
            auto int i=3;
            printf ( "\t%d ", i );
        }
        printf ( "\t%d ", i );
    }
    printf ( "t%d", i );
    return 0;
}
```

OUTPUT:

3 2 1

Register variables

These variables are stored in one of the machine's register and are declared using keyword *register*.

Example

```
register int count;
```

- ✓ Since register access are much faster than a memory access keeping frequently accessed variables in the register lead to faster execution of program.
- ✓ Use register storage class for only those variables that are being used very often in a program (loop counters).

Properties of external variable are as under:

Storage	CPU registers
Initial value	Garbage value
Scope	Local to the block in which the variable is defined.
Life	Till the control remains within the block in which the variable is defined

Example :

```
#include<stdio.h>
```

```
void main( )
```

```
{
```

```
    auto int i ,n,sum=0;
```

```
    printf("Enter a number:");
```

```
    scanf("%d",&n);
```

```
    for ( i = 1 ; i <= n ; i++ )
```

```
    {
```

```
        printf ( "\t %d ", i ) ;
```

```
        sum+=i;
```

```
    }
```

```
    printf("The sum of first %d numbers %d",n,sum);
```

```
}
```

OUTPUT:

```
Enter a number: 10
```

```
1 2 3 4 5 6 7 8 9 10
```

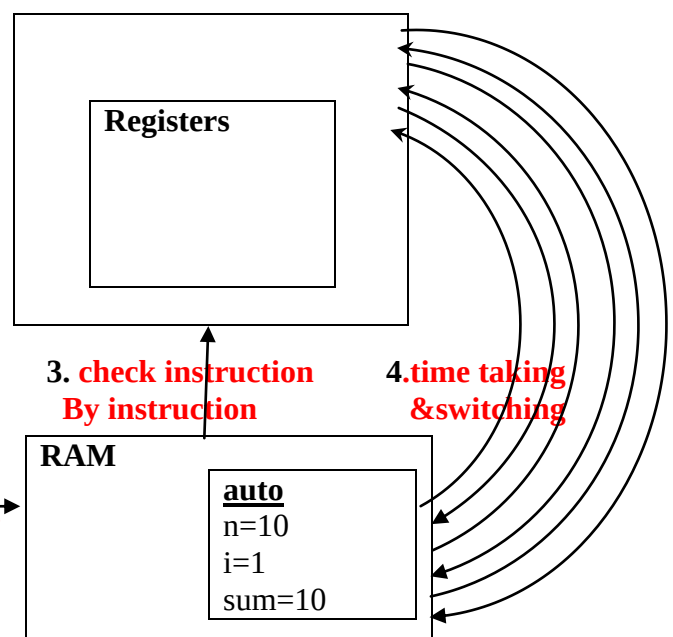
```
The sum of first 10 numbers 55
```

Memory allocation and Execution process of auto and register variables:**a) Auto variables Memory allocation & Execution process:****Hard disk (Secondary Memory)**

```
#include<stdio.h>
void main( )
{
    auto int i ,n,sum=0;
    printf("Enter
number:\n");
    scanf("%d",&n);
    for ( i = 1 ; i <= n ; i++ )
    {
        printf ( "\t %d ", i ) ;
        sum+=i;
    }
    printf("The sum of first %d
numbers %d",n,sum);
}
```

1. When you executes the program the processor will fetch the information into RAM

2. Fetching The complete program into RAM

Processor

In the above figure we observe execution process based 4 steps they are:

1. When we execute the program the processor will fetch the information into RAM.
2. Processor fetch the complete program to RAM. The auto variable memory allocation in RAM only.
3. The processor collect information instruction by instruction from the RAM. The processor executes single instruction at a time.
4. For fetching, processing, storing and updating variables and control information to one location to another(RAM to processor and processor to RAM).The processor will performs fetching and processing and updating the data instruction by instruction and returns results to RAM. In this every time processor takes instruction from RAM and executes and update the variable and returns value to RAM upto number of times. This process will occur in looping concepts. the drawback of auto variable is switching and time taking process to execute the variable repeatedly. To overcome this we go for register variables.

b) Register variable Memory allocation & Execution process:

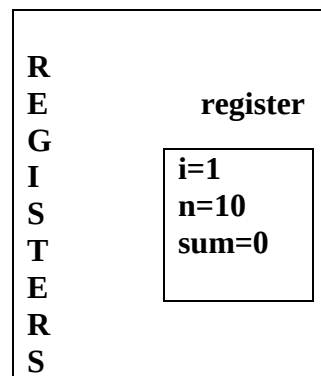
Hard Disk (Secondary Memory)

```
#include<stdio.h>
void main( )
{
    register int i ,n,sum=0;
    printf("Enter a
number:\n");
    scanf("%d",&n);
    for ( i = 1 ; i <= n ; i++ )
    {
        printf ( "\t %d ", i );
        sum+=i;
    }
    printf("The sum of first %d
numbers %d",n,sum);
}
```

1) when we execute the program the processor will fetch information into registers

2) Fetching the complete program into registers

Processor Iterators



Fetching, processing, storing & updating variables inside the processor, so the register variables executes fast.

In the above figure we observe execution process based following steps :

1. When we execute the program the processor will fetch the complete program into register.
2. Processor will fetch complete program into registers.
3. Fetching , processing, updating & storing the data inside the processor . no need to send and receive information from other places, so the register variable executes faster than other variables.

Question: In this concept, instead of all the variables (auto variable) why can't we declare register variable? Directly we can remove the auto storage class , if only maintain register storage class, it better to every program executes faster ?

Answer : Here the problem is register size is very less it may 8KB to 16KB, this much memory is always executes instruction not to store the variable data. So the register storage class always prefer to executes and storing looping programs for faster execution.

Static variables

The value of static variables persists until the end of the program. It is declared using the *static* keyword.

Example: static int x;

- ✓ Static variables are initialized only once, when the program is compiled.
- ✓ Use *static* storage class only if you want the value of a variable to persist between different function calls.

Properties of static variable are as under:

Storage	Memory
Initial value	Zero
Scope	Local to the block in which the variable is defined
Life time	Value of the variable persists b/w different function calls. Ending of program

Example

```
#include<stdio.h>
void increment();
int main( )
{
    increment();
    increment();
    increment();
    return 0;
}
void increment()
{
    static int i ;
    printf ( "%d\t", i );
    i = i + 1 ;
}
```

OUTPUT:

0 1 2

Difference between auto and static storage classes:

	auto	static
<pre>#include<stdio.h> void main() { fun(); fun(); fun(); } void fun() { static auto int a=10; printf(“%d\t”,a+=2); }</pre>	a=10; a=12; prints 12 a=10; a=12; prints 12 a=10; a=12; prints 12	a=10; a=12; prints 12 a=12; a=14; prints 14 a=14; a=16; prints 16
✓ If we declare auto keyword then the value of variable again initialized to every function call. The life time of variable particular block only. The value is not persists different function calls. ✓ Output: 12 12 12	✓ If we declare static keyword the value of variable persists to every function. the life time of value is ending of the program. ✓ Output: 12 14 16	

External Variables

- ✓ The extern keyword is used with a variable to inform the compiler that variable is declared somewhere else.
- ✓ The extern storage class is used to give a reference of a global variable that is visible to all program files.
- ✓ It is similar to global variable.
- ✓ These variables are active and alive throughout the entire program.
- ✓ The keyword *extern* used to declare these variables.
- ✓ Unlike local variables they are accessed by any function in the program.
- ✓ In case local and global variable have the same name, the local variable will have precedence over the global one.

Properties of external variable are as under:

Storage	Memory
Initial value	Zero
Scope	Global and file scope.
Life time	Until the ending of the program's execution.

Example1

```
#include<stdio.h>
void main( )
{
    int x=24;
    extern int y;
    printf("x=%d\n",x);
    printf("y=%d\n",y);
}
y=10;
```

OUTPUT:

```
x=24
y=10
```

Example2

```
file1.c
#include<stdio.h>
#include"file2.c"
void main( )
{
    extern int x;
    extern int y;
    printf("x=%d\n",x);
    printf("y=%d\n",y);
}
```

OUTPUT:

```
x=24
y=1224
```

```
file2.c
#include<stdio.h>
x=24;
int y =1224;
```

Example3**File3.c**

```
#include<stdio.h>
#include"file4.c"
void main( )
{
    extern int a;
    printf("%d\n",a);
    fun( );
    printf("%d\n",a);
}
```

OUTPUT:

```
7
8
8
```

File4.c

```
#include<stdio.h>
int a=7;
void fun( )
{
    a++;
    printf("%d\n",a);
}
```

Example 4:

Write a program to arithmetic operators using extern keyword . take each operator is one function. Functions are declare inside file2.

File5.c

```
#include<stdio.h>
#include"file6.c"
void add( );
void sub( );
void mul( );
void div( );
void rem( );
void main( )
{
    extern int a,b;
    add( );
    sub( );
    mul( );
    div( );
    rem( );
}
```

OUTPUT:

```
the sum of 20 & 10 is 30
the sub of 20 & 10 is 10
the mul of 20 & 10 is 200
the div of 20 & 10 is 2
the mod of 20 & 10 is 0
```

File6.c

```
#include<stdio.h>
a=20;
b=10;
void add( )
{
    printf("the sum of %d & %d%d\n",a,b,a+b);
}
void sub( )
{
    printf("the sub of %d & %d%d\n",a,b,a-b);
}
void mul( )
{
    printf("the mul of %d & %d%d\n",a,b,a*b);
}
void div( )
{
    printf("the div of %d & %d%d\n",a,b,a/b);
}
void rem( )
{
    printf("the mod of %d & %d%d\n",a,b,a%b);
}
```

Example 5:

Write a program to access global variable in All functions within the program.

```
#include<stdio.h>
voidinc();
voiddec();
int i;           //i is a global variable
int main( )
{
    printf("Intial value is %d ", i);
    inc();
    inc();
    dec();
    dec();
    return 0;
}
voidinc()
{
    i++;
    printf("\nOn incrementing:%d ",i);
}
voiddec()
{
    i--;
    printf("\nOn decrementing:%d ",i);
}
```

OUTPUT

```
Initial value is 0
On incrementing: 1
On incrementing: 2
On decrementing: 1
On decrementing: 0
```

4.4 Recursion

Recursion means call by itself. A function that calls itself repetitively is known as a *recursive function*. Simply calling a function from definition of a same function, this technique is known as **recursion**. The function calls itself repetitively until certain condition is satisfied.

- ✓ Generally to executes functions in application, inside RAM ,the memory is allocated known as stack memory.
- ✓ To execution of every function/blocks some memory allocates inside the stack that is called frames.

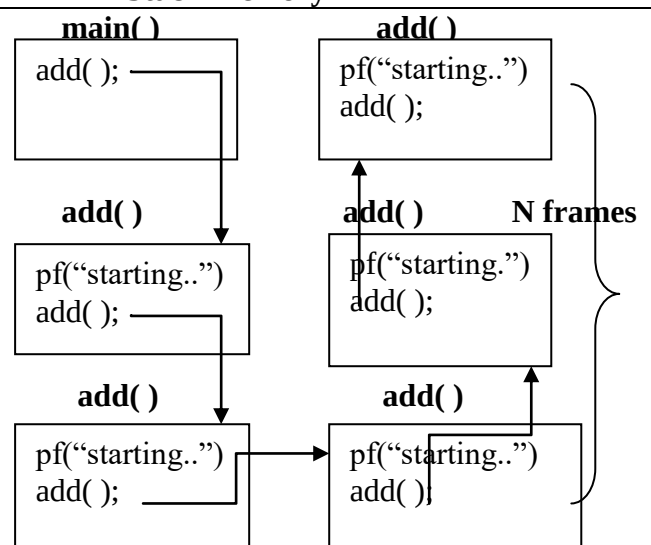
Example:

```
void main( )
{
    add( );
}
void ( )
{
    printf("Staring of the sub program");
    add( );
    printf("Staring of the sub program");
}
```

Note:

- ✓ The program is terminates abnormally because occurs runtime error, i.e out of memory error.
- ✓ Here the add () function call by itself infinite times because we are not specify the termination condition, so the add functions runs continuously at certain position the stack will be full. There no space available for calling that function again, then compiler generate runtime error i.e out of memory.

Stack Memory



- ✓ In the above program flow of control only forward direction, it should back to control where the execution started.
- ✓ To stop the infinite recursion, you need to insert one base case in program. The base condition stops the recursion.
- ✓ While working with recursive functions, you should follow some steps while writing the logic of program because the control which order to goes forward that order come control to backward direction.
- ✓ You should follow the following types of statement while you writing the program.

The recursive function has following types of statements:

1. A statement or *condition* to determine if the function is calling itself again.
2. A *function call* which should have argument.
3. Conditional statement(if-else). **or** Base Case
4. A *return* statement.

Example: C Program that calculates factorial of a given number using recursion

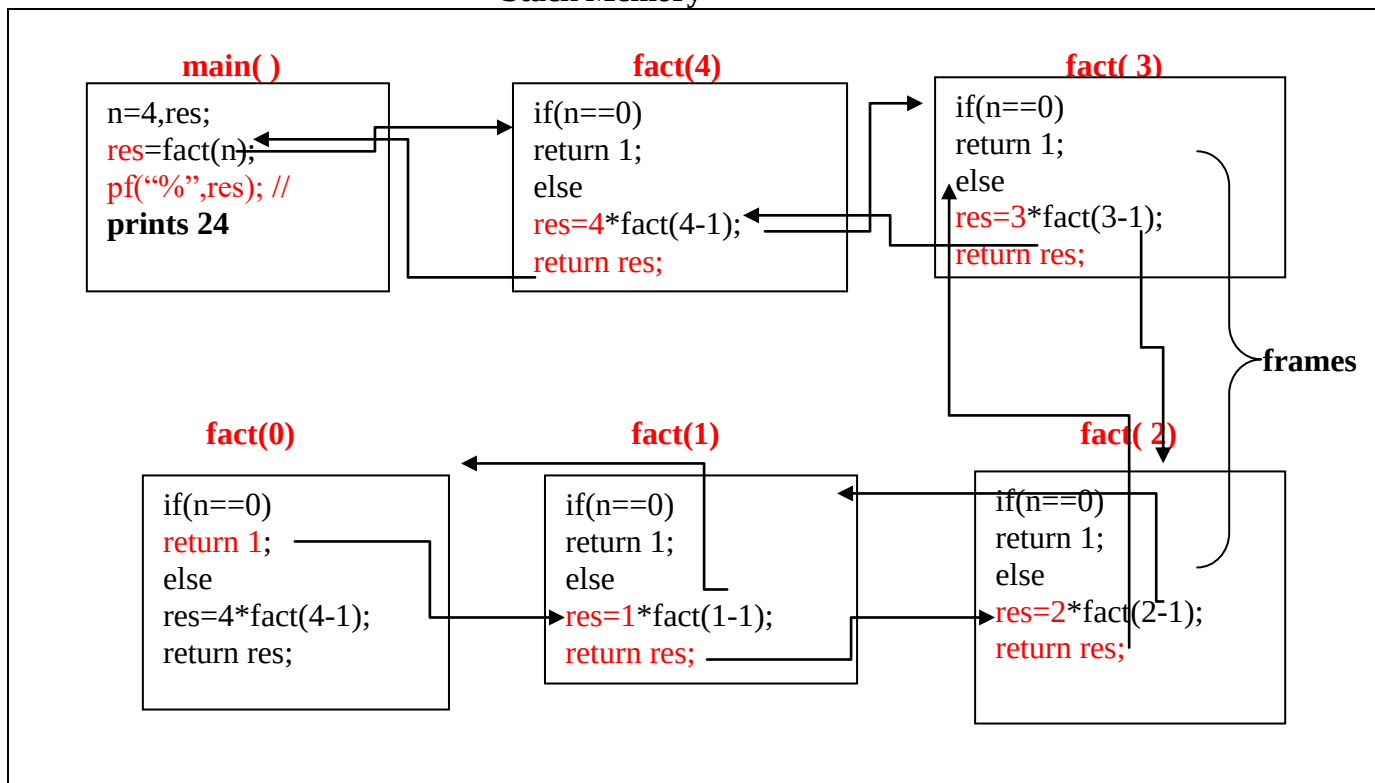
```
#include <stdio.h>
long int factorial(int);
int main()
{
    int n;
    long int res;
    printf("Enter a number:");
    scanf("%d",&n);
    f=fact(n);
    printf("Factorial of %d is %ld\n",n,res);
    return 0;
}
long int fact(int n)
{
    int res;
    if(n==0) // Base case
        return 1;
    else
        res=n*fact(n-1);
    return res;
}
```

OUTPUT:

Enter a number: 4
Factorial of 4 is 24

Flow of execution process & Memory allocation of the above program

Stack Memory



Note:

- ✓ In the above program, red colors statements executes in backward control process.
- ✓ Every function execution compiler allocates separate frames in stack memory

Types of Recursion :

The following are the types of Recursions.

1. Direct Recursion
2. Indirect Recursion
3. Tail Recursion
4. Non-tail Recursion
5. Linear Recursion
6. Tree Recursion

Direct Recursion:

- ✓ A function is called indirect recursive if it calls the same function again.

Example:

```
void fun( )
{
    // statements
    fun( );
    // statements
}
```

Indirect Recursion:

- ✓ A function1 is called indirect recursive if it calls another function2 and then function2 calls function1 directly or indirectly.

Structure:

```
void fun1( )
{
    // statements
    fun2( );
    // statements
}
void fun2( )
{
    // statements
    fun1( ); → Indirect function call of fun1
    // statements
}
```

Example:

Write program to print 1 to 10 numbers in such a way that if numbers is odd then convert to even and when numbers is even then convert to odd.

```
#include<stdio.h>
void odd();
void even();
int n=1;
void odd()
{
    if(n<=10)
    {
        printf("%d\t",n+1);
        n++;
        even();
    }
}
```

```

        return;
    }
    void even()
    {
        if(n<=10)
        {
            printf("%d\t",n-1);
            n++;
            odd();
        }
        return;
    }
    int main( )
    {
        odd();
        return 0;
    }

```

OUTPUT:
2 1 4 3 6 5 8 7 10 9

Tail Recursion & Non-tail Recursion :

Tail Recursion	Non-tail Recursion
✓ A recursive function said to the tail recursive , if the recursive call is last thing (statement) done by the function. These is no need to keep record of the previous state. Example 1: #include<stdio.h> int fun(int n) { if(n==0) return 0; else printf("%d/t",n); return fun(n-1); // function call is last statement of function. } int main() { fun(7); return 0; } OUTPUT: 7 6 5 4 3 2 1	✓ If the recursive call is not the last thing (statement) done by the functions, after returning back, there is some statements are left. Example 1: #include<stdio.h> int fun(int n) { if(n==0) return 0; else fun(n-1); // function call is not last statement. printf("%d",n); } int main() { fun(7); return 0; } OUTPUT: 1 2 3 4 5 6 7

Example 2: Write a program to print the Nth number of the fibonacci series.

```
#include<stdio.h>
int fib(int,int,int);
int fib(int n,int a,int b)
{
    if(n==1)
    {
        return a;
    }
    if(n==1)
    {
        return b;
    }
    return fib(n-1,b,a+b); // tail recursion.
}
int main()
{
    int n,a=0,b=1;
    printf("Enter a number :");
    scanf("%d",&n);
    printf("The fibonacci series %d ends
        with %d ",n,fib(n,a,b));
    return 0;
}
```

OUTPUT:

Enter a number: 10
The fibonacci series 10 ends with 34

Example 2:

```
#include<stdio.h>
int fun( int n);
int fun(int n)
{
    if(n==0)
        return 0;
    else
        return 1+fun(n/2); // it is not a tail
                                recursion, to
                                clear understanding
                                refer following note
}

int main()
{
    printf("%d",fun(8));
    return 0;
}
```

OUTPUT:

4

Note : In the above program, fun() having a last statement i.e **return 1+fun(n/2)**. whenever reaches that statement the function calls itself, after completion of the function execution then the resultant value is adds with 1, here the return statement having expression not only function call. So, In tail recursion , must ends with only function call not in the form of expression.

Linear Recursion:

- ✓ A recursive function is said to be linearly recursive when the pending operation does not make another recursive call function.

Examples:

1. return n*fact(n-1) ; → right
2. res=n*n-1+add(20,30); → right
3. return fib(n-1)+fib(n-2); → wrong
4. return n*fib(n-1)+fib(n-2); → wrong

Note:

- ✓ In the above Examples, 1 & 2 having only single recursive function call ,here operation pending doesn't make another recursive function call. the pending operations that performs with variables

not function calls.

- ✓ In 3 & 4 having more than one function call, here operations pending for another recursive function call, the operation waiting for another recursive function call. but according to linear recursion, the pending operations doesn't wait for another recursive function call.

Example

```
int fact(int n)
{
    if(n==0)
        return 1;
    else
        return n*fact(n-1);
}
```

→ only single recursive function call, this pending operation doesn't wait for another recursive function call.

Note : If we have more than one recursive function call in our expression then we have to know about Tree recursion concept.

Tree Recursion:

- ✓ A recursive function is said to be non- linearly (tree) recursive if the pending operation makes another recursive function call.
- ✓ Tree recursion also called as non-linear recursive functions.
- ✓ If we include more than one recursive function call in our expression is known as tree recursion.

Examples:

1. return fib(n-1)+fib(n-2);
 2. return n*fib(n-1)+fib(n-2);
- } **(Tree recursions)** more than one call recursive function call in expressions.

Note:

- ✓ In the above two examples, the operations waiting for another recursive function call, then only complete that operation successful.
- ✓ In this, fib(n-2) is called successfully that functions returns some value but to complete that operation wait for another recursive function call i.e fib(n-1). After completion of fib(n-1) function it returns some value then evaluate the total expression (i.e adds fib(n-1) returned value and fib(n-2) returned value.).

Example 1:

Write a program to print the n+1 th number of the fibonacci series using tree recursion.

```
#include<stdio.h>
int fib(int x);
int main()
{
    int n;
    printf("enter N value");
    scanf("%d",&n);
    printf("The n+1th number in fibonacci
           series is %d\t",fib(n));
    return 0;
}
int fib(int n)
{
    if(n==0)
        return 0;
    else if(n==1)
        return 1;
    else
        return (fib(n-1)+fib(n-2));
}
```

OUTPUT:

Enter N value : 10
The N+1th number in Fibonacci series is 55

Note: For better understanding the above program, observe the flow of execution in below figure.

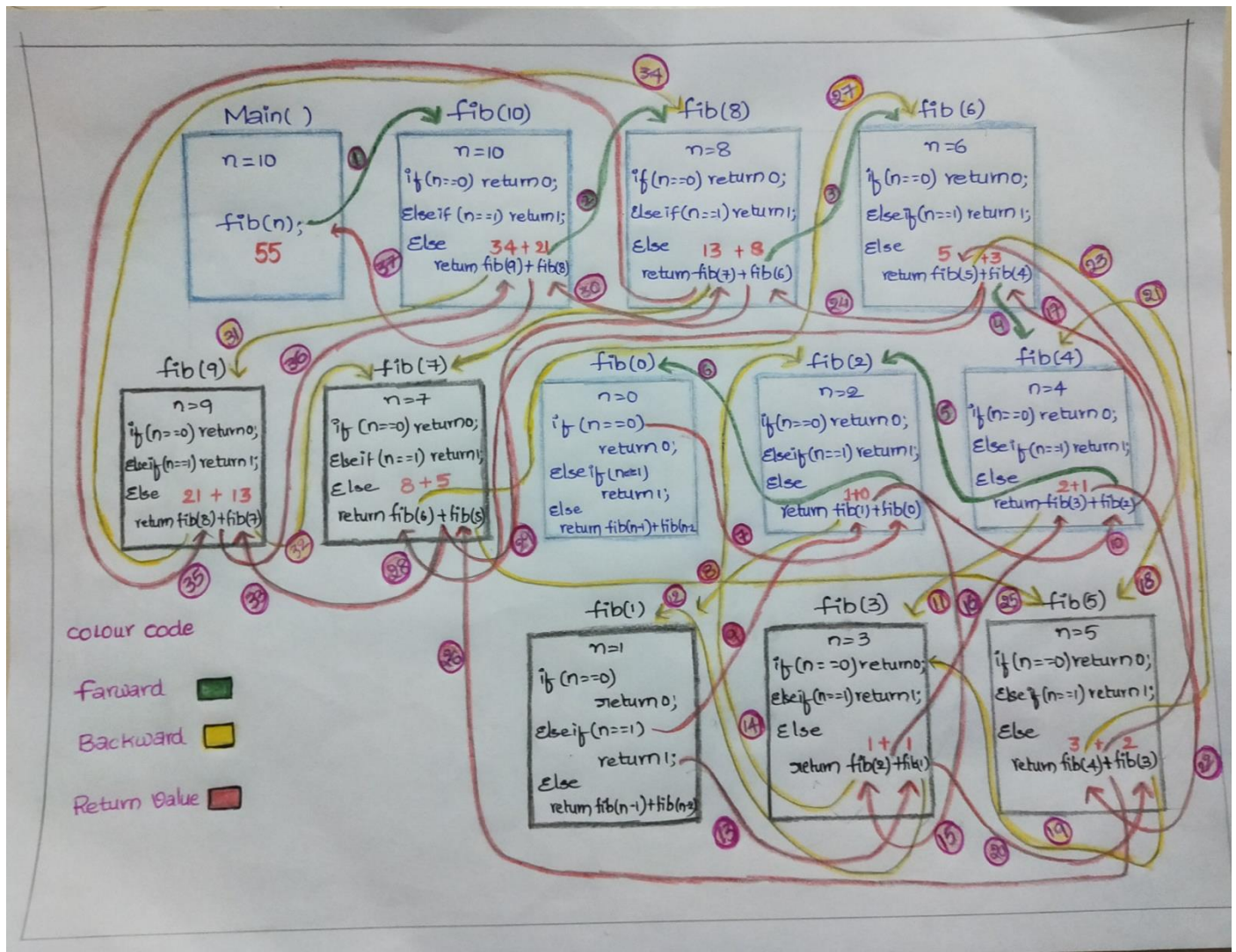
Example 2:

Write a program to print all n numbers of fibonacci series using tree recursion.

```
#include<stdio.h>
int fib(int x);
int main()
{
    int n,i;
    printf("enter a number");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("%d\t",fib(i));
    }
    return 0;
}
int fib(int n)
{
    if(n==0)
        return 0;
    else if(n==1)
        return 1;
    else
        return (fib(n-1)+fib(n-2));
}
```

OUTPUT:

Enter a number : 10
0 1 1 2 3 5 8 13 21 34

Flow of Execution for Example 1: (To print the n+1 th number of the fibonacci series)

- ✓ In the above figure , we observe flow of execution in both directions i.e forward and backward direction.
- ✓ Forward flow executes until the base condition (base case) is true, if the base case true then return value to previous calling function. Once base condition true then the flow execution is starts from backward direction.
- ✓ In this figure, we observe the three colored lines that indicates
 - Green color indicates forward flow of execution.
 - Yellow color for backward direction, in this direction also includes forward and backward flow of execution.
 - Red color indicates returns the results to calling function.

Advantages of Recursion :

- ✓ Recursive solutions often tend to be shorter & simpler than non-recursive ones.
- ✓ Code is clear and easier to use.
- ✓ Recursion works similar to the original formula to solve a problem.
- ✓ In some cases, recursion may be efficient.
- ✓ It follows a divide and conquer technique to solve a problem.

Disadvantages of Recursion :

- ✓ For some programmers and readers, recursion is a difficult concept.
- ✓ Recursion is implemented using system stack(stack memory).If the stack space on the system is limited, recursion to deeper level will be difficult to implement.
- ✓ It takes more memory and more execution time to execute as compared to non-recursive parts.
- ✓ It is difficult to find bugs, particularly while using global variable.

Difference between Recursion and Iterators :

Recursion	Iterators
1. The statement in a body of function calls the function itself.	1.Set of instructions to be repeatedly executed.
2. Top down approach to problem solving in which the original problem is divided into smaller sub-problems.	2. It follows the bottom-up approach that what is known and then constructing the solution step by step.
3. In recursive functions, only termination condition (Base case) is specified.	3. Iteration include initialization, condition , execution of statement within loop and update(++ or --) the variable.
4. If the function does not contain base condition ,it leads to infinite recursion.	4. If the condition in the iteration, statement never become false, it leads infinite iteration.
5. The stack is used to store the set of new local variables and parameters each time the function call.	5. Doesn't uses stack.
6. Slow in execution.	6. Fast in execution.
7. Recursion reduces the size of code.	7. Iteration makes the longer code.

Problems solving using Functions:**1. Write a program to Tower of Hanoi concept using recursive functions.**

```
#include<stdio.h>
void tower(int,char,char,char);
int main()
{
    int n;
    printf("enter the number of rings:");
    scanf("%d",&n);
    printf("The sequence of moves involved in the tower of hanoi are:\n");
    tower(n,'A','C','B');
    return 0;
}
void tower(int n,char source,char dest,char spare)
{
    if(n==1)
```

```
{
printf("\n Move from %c to %c",source,dest);
}
else
{
tower(n-1,source,spare,dest);
tower(1,source,dest,spare);
tower(n-1,spare,dest,source);
}
}
```

OUTPUT:

Enter the number of ring: 3
The sequence of moves involved in the tower of hanoi are:
Move from A to C
Move from A to B
Move from A to B
Move from A to C
Move from B to A
Move from B to C
Move from A to C

2. Write program to solve $f(n)=1+(2*3)+(4*5*6)+(7*8*9*10) \dots n$ using recursion, here n is number of sets in the expressions.

```
#include<stdio.h>
int seriesSum(int,int,int);
int seriesSum(int cal,int cur1,int n)
{
    int i,cur2=1;
    if(cur1==n+1)
    {
        return 0;
    }
    for(i=cal;i<cal+cur1;i++)
    {
        cur2=cur2*i;
    }
    return cur2+seriesSum(i,cur1+1,n);
}

int main()
{
    int n;
    printf("Enter no of sets to be evaluate:");
    scanf("%d",&n);
    printf("%d\n",seriesSum(1,1,n));
    return 0;
}
```

OUTPUT:

Enter no of sets to be evaluate : 3
127

3. Find output for following program

```
#include<stdio.h>
void fun(int,int);
void main()
{
    int n;
    printf("enter a number:");
    scanf("%d",&n);
    fun(--n, n++);
    fun(++n, n--);
    printf("%d\n",n);
}
void fun(int x,int y)
{
    printf("%d \n  %d",x++ ,y--);
}
```

OUTPUT:

```
24
24
24
24
24
```

4. Write a program to find GCD of two numbers using recursion.

```
#include<stdio.h>
int Gcd(int,int);
int main()
{
    int num1,num2,res;
    printf("\n Enter two numbers:");
    scanf("%d %d",&num1,&num2);
    res=Gcd(num1,num2);
    printf("\n GCD of %d and %d is %d",num1,num2,res);
    return 0;
}
int Gcd(int x,int y)
{
    int rem;
    rem=x%y;
    if(rem==0)
        return y;
    else
        return (Gcd(y,rem));
}
```

OUTPUT:

```
Enter two numbers:
10
5
GCD of 10 and 5 is 5
```

Solve the following programs

1. Write a program to find sum of the series $1+1/2+1/3+1/4+.....1/n$
2. The value of j at the end of the execution of the following program (storage classes)
(GATE CS2000 Question)

```
#include<stdio.h>
int incr(int i)
{
    static int count =0;
    count=count+i;
    return (count);
}
main()
{
    int i,j;
    for(i=0;i<=4;i++)
        j=incr(i);
}
```

3. What is the output of the following program

```
main()
{
    int x=20;
    int y=10;
    swap(x,y);
    printf("%d %d",y,x+2);
}

swap(int x,int y)
{
    int temp;
    temp=x;
    x=y;
    y=temp;
}
```
